

🕒 Cập nhật tháng 8 năm 2024

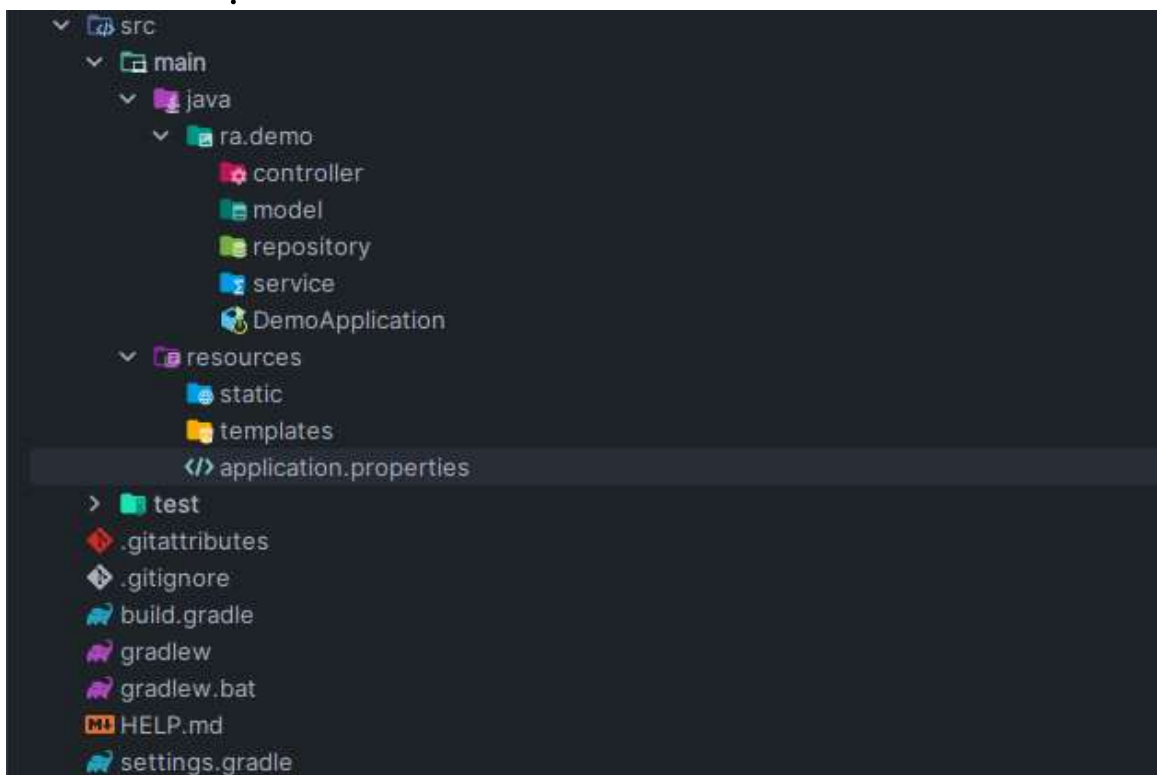
[Bài đọc] Xây dựng các API cơ bản với Spring Boot

1. Giới thiệu về REST API

- **REST (Representational State Transfer)** là một kiến trúc xây dựng dịch vụ web, cho phép client và server giao tiếp qua HTTP
- Một REST API cơ bản sử dụng các **HTTP method**:
 - **GET** → Lấy dữ liệu (Read)
 - **POST** → Thêm dữ liệu mới (Create)
 - **PUT/PATCH** → Cập nhật dữ liệu (Update)
 - **DELETE** → Xóa dữ liệu (Delete)
- 👉 Tức là chúng ta có thể xây dựng đầy đủ **CRUD API** (Create, Read, Update, Delete) với Spring Boot

2. Các bước xây dựng API cơ bản với Spring Boot

- Tạo dự án Spring Boot: [Link hướng dẫn tạo dự án](#)
- Cấu trúc dự án cơ bản



- Tạo Entity (Model)

```
package ra.demo.model;

import jakarta.persistence.*;

@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // Getters và Setters
}
```

- Tạo Repository

```
package ra.demo.repository;

import com.example.demo.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

}
```

- Tạo Service
- Tạo Controller (API)

```
package com.example.demo.controller;

import com.example.demo.model.User;
import com.example.demo.service.UserService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    // GET all users
    @GetMapping
    public List<User> getAllUsers() {
        return userService.getAll();
    }

    // GET user by id
    @GetMapping("/{id}")
    public User getUserById(@PathVariable Long id) {
        return userService.getById(id);
    }

    // POST create new user
    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.save(user);
    }

    // PUT update user
    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id, @RequestBody User user) {
        return userService.update(id, user);
    }

    // DELETE remove user
    @DeleteMapping("/{id}")
    public String deleteUser(@PathVariable Long id) {
        userService.delete(id);
        return "User deleted successfully!";
    }
}
```

- Cấu hình application.properties

```
1  spring.application.name=demo
2
3  # server port
4  server.port=8080
5
6  # H2 database
7  spring.datasource.url=jdbc:mysql://localhost:3307/db_example
8  spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
9  spring.datasource.username=root
10 spring.datasource.password=123456
11
12 spring.jpa.hibernate.ddl-auto=update
13 spring.jpa.show-sql=true
```

- Kiểm thử API

- Chạy ứng dụng (**DemoApplication.java**)

- Dùng **Postman** hoặc **cURL** để test:

- **GET**: <http://localhost:8080/api/users> → lấy danh sách

- **POST**: <http://localhost:8080/api/users>

- body:

```
{
  "name": "Alice",
  "email": "alice@example.com"
}
```

- **PUT**: <http://localhost:8080/api/users/1>

- body:

```
{
  "name": "Alice Updated",
  "email": "alice.new@example.com"
}
```

- **DELETE**: <http://localhost:8080/api/users/1>

3. Tóm tắt

- Với **Spring Boot**, việc xây dựng API cơ bản rất nhanh nhờ cấu trúc rõ ràng: **Controller - Service - Repository - Model**

- Ta có thể tạo **CRUD API** chỉ với vài bước:

- Định nghĩa **Entity** (Model)

- Tạo **Repository** (JpaRepository)

- Viết **Service** (logic)
- Viết **Controller** (API endpoint)
- Có thể kiểm thử bằng **Postman** hoặc trình duyệt (đối với GET API)

Danh sách các bài học

